

CI/CD Pipeline & Compatibility Guide

Camera Firmware (Eco-Series-Kitty) & MQTT Broker Server

Author:

Rahul Vadhiya

Date:

March 2026

Client Repo:

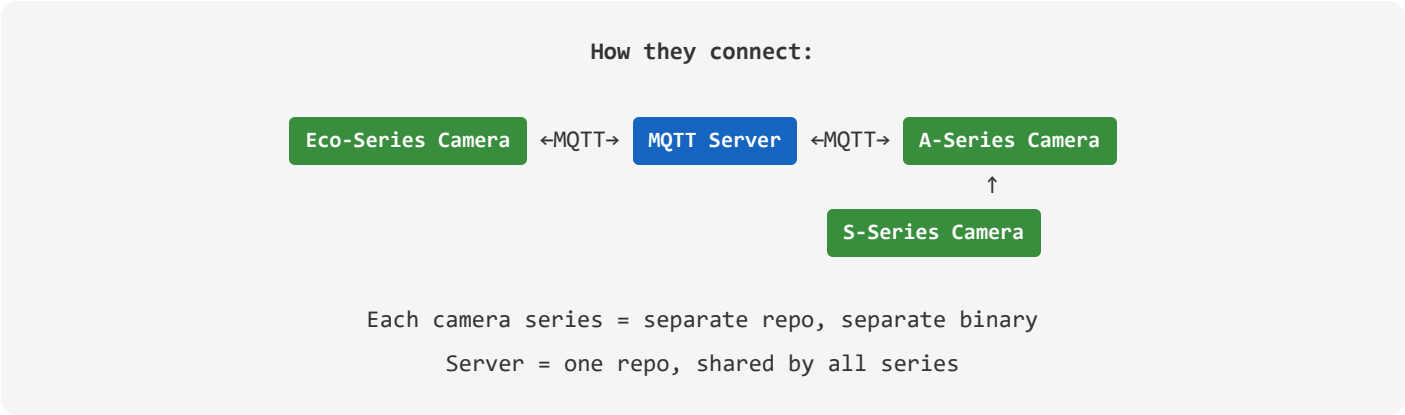
github.com/Rahul-ambiplatforms/Eco-Series-Kitty

Server Repo:

github.com/Rahul-ambiplatforms/mqtt-server

1. Repository Architecture

	Eco-Series-Kitty (Camera Firmware)	mqtt-server (MQTT Broker)
Language	C (cross-compiled to ARM)	Node.js
Version stored in	kitty/Augentix/AugenTix/config.json "firmware": "A_Series_X.Y.Z"	package.json "version": "X.Y.Z"
CI workflow	.github/workflows/release.yml	.github/workflows/validate.yml
Release artifact	Augentix.tar.gz (downloadable)	None (deploy via git pull)
Deploy method	OTA to camera (MQTT case 36)	SSH → git pull → npm install → restart
Serves	Eco Series cameras only	ALL camera series (Eco, A, S)



2. Camera Firmware CI/CD Pipeline

2.1 When does it trigger?

You push to	CI triggers?	What happens
mqtt-client/	Yes	Full build + release

kitty/	Yes	Full build + release
docs/	No	Nothing
toolchain/	No	Nothing
Commit with [skip ci]	No	Nothing (prevents infinite loop)
Manual (Actions tab)	Yes	Select bump type → build + release

2.2 Pipeline Flow

Step 1: DETECT BUMP TYPE (from commit message) |—"feat" or "feature" in message → **MINOR** bump (0.12.0 → 0.13.0) |—"fix", "bug", "hotfix" in message → **PATCH** bump (0.12.1 → 0.12.2) |— anything else → **PATCH** bump **Step 2: BUMP VERSION** Read config.json → "A_Series_0.12.1" Apply bump → "A_Series_0.12.2" Write back to config.json **Step 3: SET TOOLCHAIN PERMISSIONS** chmod +x on bin/, libexec/, arm-augentix/bin/ (git doesn't preserve Linux execute bits from Windows) **Step 4: BUILD ARM BINARY** export PATH=toolchain/bin:\$PATH make -C mqtt-client/ clean all Output: bin/MQTT_vcamclient_Augentix **Step 5: VERIFY BINARY** file MQTT_vcamclient_Augentix → must contain "ARM" Fails pipeline if not ARM 32-bit ELF **Step 6: ASSEMBLE KITTY PACKAGE** Copy binary into kitty/Augentix/AugenTix/ chmod +x on binary, P2Pambicam, ambicam.sh **Step 7: CREATE AUGENTIX.TAR.GZ** tar -czf Augentix.tar.gz (the OTA package) **Step 8: COMMIT + TAG + PUSH** git commit "Release v0.12.2 [skip ci]" git tag v0.12.2 git push --follow-tags **Step 9: GENERATE CHANGELOG** Get all commits since last tag Categorize into: New Features / Bug Fixes / Other Detect what changed: Client? Kitty? **Step 10: CREATE GITHUB RELEASE** Title: "Kitty Firmware v0.12.2" Body: Changelog + What Changed table + Package Contents Asset: Augentix.tar.gz (downloadable)

2.3 Example: Push a bug fix

```
# Edit the code
nano mqtt-client/HTTP.c

# Commit with "fix" keyword
git add mqtt-client/HTTP.c
git commit -m "fix: resolve null pointer in PTZ handler"
git push

# CI automatically:
# 1. Detects "fix" → PATCH bump
# 2. 0.12.1 → 0.12.2
# 3. Builds ARM binary
# 4. Packages Augentix.tar.gz
# 5. Tags v0.12.2
# 6. Creates GitHub Release with changelog
```

2.4 Example: Push a new feature

```
git commit -m "feat: add case 81 for speaker volume control"
git push

# CI: detects "feat" → MINOR bump → 0.12.2 → 0.13.0 → builds → releases
```

2.5 Example: Update startup script

```
git add kitty/Augentix/AugenTix/ambicam.sh
git commit -m "update retry timeout in ambicam.sh"
git push

# CI: default PATCH → 0.13.0 → 0.13.1 → builds → releases
```

3. MQTT Server CI/CD Pipeline

3.1 When does it trigger?

Event	What runs
Push to main	Validate → Release (tag + changelog + GitHub Release)
Pull Request to main	Validate only (no release)
Commit with [skip ci]	Nothing
Manual (Actions tab)	Validate → Release with selected bump type

3.2 Pipeline Flow (Two Jobs)

JOB 1: VALIDATE (runs on push + PR) |— Install Node.js 18 + npm dependencies |— Syntax check: node -c mqtttcode.js |— Load all 14 Mongoose models – fail if any error |— Load commonConfig – verify 10+ case types exist ↓ must pass **JOB 2: RELEASE** (runs on push to main only, NOT on PR) **Step 1: DETECT BUMP TYPE** |— "feat" → MINOR (1.2.0 → 1.3.0) |— "fix" → PATCH (1.2.0 → 1.2.1) |— other → PATCH **Step 2: BUMP VERSION IN PACKAGE.JSON** "version": "1.2.0" → "1.2.1" **Step 3: GENERATE CHANGELOG** Commits since last tag → Features / Bug Fixes / Other **Step 4: COMMIT + TAG + PUSH (atomic)** git commit "Release v1.2.1 [skip ci]" git tag v1.2.1 git push --follow-tags ← commit + tag pushed together **Step 5: CREATE GITHUB RELEASE** Title: "MQTT Server v1.2.1" Body: Changelog + Compatible Client Versions + Deployment steps

3.3 Example: Push a server bug fix

```
cd mqttt-server-repo/
nano mqtttcode.js

git add mqtttcode.js
```

```
git commit -m "fix: handle null payload in case 37"
git push

# CI automatically:
# 1. Validates syntax + models
# 2. Detects "fix" → PATCH bump
# 3. 1.2.0 → 1.2.1
# 4. Tags v1.2.1
# 5. Creates GitHub Release with changelog

# Then deploy to server manually:
ssh user@prong.arcisai.io
cd /path/to/mqtt-server && git pull && npm install --production
pm2 restart mqtt-server
```

4. Version Bump Cheat Sheet

What you did	Commit message	Bump	Example
Fixed a crash	fix: null pointer in onConnectionLost	Patch	0.12.1 → 0.12.2
Fixed a memory leak	bugfix: free cJSON in get_system_info	Patch	0.12.2 → 0.12.3
Added new MQTT case	feat: add case 81 for speaker volume	Minor	0.12.3 → 0.13.0
Added PTZ preset feature	feature: support PTZ preset tour	Minor	0.13.0 → 0.14.0
Changed PTZ speed	update PTZ speed to 5	Patch	0.14.0 → 0.14.1
Updated startup script	update ambicam.sh retry logic	Patch	0.14.1 → 0.14.2
Security hotfix	hotfix: validate OTA URL	Patch	0.14.2 → 0.14.3
Server: added new case	feat: handle case 81 on server	Minor	1.2.1 → 1.3.0
Server: fixed decrypt	fix: decrypt password before send	Patch	1.3.0 → 1.3.1

Important: Keywords are case-insensitive and can appear anywhere in the commit message.

"fix: something", "Fixed the bug", "This is a bugfix" — all trigger patch.

"feat: new thing", "Added feature for PTZ" — all trigger minor.

5. Compatibility Management

5.1 The Golden Rule

NEVER change the JSON format of an existing case number.

Always add a NEW case number for new functionality.

5.2 Why it works

Scenario	What happens	Result
New server sends case 81 to old client	Old client has no case 81 handler → hits default: in switch	Prints "Unhandled case 81" → ignored safely
New client expects case 81 from old server	Old server doesn't send case 81	Client uses defaults → works
Server changes case 80 JSON format	Client parses wrong fields	BREAKS — never do this!
Server adds new field to case 80 JSON	Client ignores unknown fields (cJSON skips them)	Safe — additive changes are OK

5.3 Compatibility Matrix

Both repos maintain a COMPATIBILITY.md file:

Client repo (Eco-Series-Kitty/COMPATIBILITY.md):

Client Version	Min Server Version	Cases Added
0.10.7	1.0.0	0-78 (base)
0.12.0	1.1.0	80 (credential sync)

Server repo (mqtt-server/COMPATIBILITY.md):

Server Version	Min Client Version	Cases Added
1.0.0	any	0-78 (base)
1.1.0	0.12.0	80 (credential sync)
1.1.1	0.12.0	bugfix: decrypt password

5.4 When adding a new case (e.g., case 81)

```
# Step 1: Add handler in SERVER first
cd mqtt-server-repo/
# Edit mqtttcode.js - add case '81' handler
# Edit utils/commonConfig.js - add MSG_TYPE_SPEAKER_VOLUME: '81'
# Edit COMPATIBILITY.md - add row: Server 1.3.0 | Min Client 0.13.0 | case 81
git commit -m "feat: add case 81 handler for speaker volume"
git push
# CI auto-releases server v1.3.0
# Deploy server: ssh → git pull → restart

# Step 2: Add handler in CLIENT
cd Eco-Series-Kitty/
```

```
# Edit mqtt-client/HTTP.c — add case 81 in messageArrived switch
# Edit COMPATIBILITY.md — add row: Client 0.13.0 | Min Server 1.3.0 | case 81
git commit -m "feat: add case 81 for speaker volume control"
git push
# CI auto-releases client v0.13.0 with Augentix.tar.gz

# Step 3: OTA push to cameras
# Download Augentix.tar.gz from GitHub Release
# Push to cameras via MQTT case 36
```

Always deploy server FIRST, then client.

Server must be ready to handle new cases before cameras start sending them.

6. Deployment Workflow

6.1 Deploy server update

```
# On your server machine (prong.arcisai.io)
cd /path/to/mqtt-server
git pull origin main
npm install --production
pm2 restart mqtt-server

# Verify
pm2 logs mqtt-server --lines 20
# Should see: "MongoDB (MQTT) connected..." and "Aedes MQTT Broker started"
```

6.2 Deploy camera firmware

```
# Option A: OTA via MQTT (production — all cameras)
# 1. Upload Augentix.tar.gz to OTA server
# 2. Send MQTT command to camera:
mosquitto_pub -h prong.arcisai.io -p 1883 -u Torque -P 'Raptor@0' \
  -t "torque/rx/CAMERA-SN/36" \
  -m "https://ota-server.com/firmware/Auentix.tar.gz"

# Option B: Manual deploy (single test camera)
# 1. SCP the binary to camera
scp bin/MQTT_vcamclient_Auentix root@camera-ip:/mny/mtd/ipc/ambicam/
# 2. Restart on camera
killall MQTT_vcamclient_Auentix && /mny/mtd/ipc/ambicam/ambicam.sh &
```

7. Infinite Loop Prevention

Problem: CI commits a version bump → triggers CI again → commits again → infinite loop.

Solution: Both pipelines use `[skip ci]` in auto-commit messages.

Normal flow: You push "fix: something" → CI runs → CI commits "Release v0.12.2 [skip ci]" → CI sees [skip ci] → **STOPS Without [skip ci]:** You push → CI runs → CI commits → CI runs → CI commits → CI runs → ... **FOREVER**

8. GitHub Release Notes Format

8.1 Camera firmware release

```
## Kitty Firmware v0.13.0

Firmware: A_Series_0.13.0
Release type: feature
Platform: ARM 32-bit (Augentix / uClibc)
Date: 2026-03-26
Previous version: v0.12.2

---

### What Changed
| Component | Changed |
|-----|-----|
| MQTT Client (C binary) | Yes |
| Kitty Package (scripts/config) | No |

---

### Changelog

#### New Features
- feat: add case 81 for speaker volume control

#### Bug Fixes
- fix: resolve null pointer in PTZ handler

#### Other Changes
- update websocket frame size limit

---

### Package Contents
| File | Description |
|-----|-----|
| MQTT_vcamclient_Augentix | MQTT camera client binary |
| P2Pambicam | P2P tunneling client |
| ambicam.sh | Startup & process monitor script |
| config.json | Firmware version & MQTT config |
| lib*.so.* | Shared libraries |
```

```
### Deployment
1. Download Augentix.tar.gz from assets below
2. Upload to OTA server
3. Push to cameras via MQTT case 36
```

8.2 Server release

```
## MQTT Server v1.3.0

Version: 1.3.0
Release type: feature
Previous version: v1.2.1
Date: 2026-03-26

---

### Changelog

#### New Features
- feat: add case 81 handler for speaker volume

#### Bug Fixes
- fix: handle null payload in case 37

---

### Compatible Client Versions
| Camera Series | Min Client Version |
|-----|-----|
| Eco Series | 0.13.0 |
| A-Series | TBD |
| S-Series | TBD |

### Deployment
1. Pull latest code: git pull origin main
2. Install deps: npm install --production
3. Restart: pm2 restart mqtt-server
```

9. Quick Reference Card

Task	Command	What CI does
Fix client bug	git commit -m "fix: description" && git push	Patch bump → build → release
New client feature	git commit -m "feat: description" && git push	Minor bump → build → release
Fix server bug	cd mqtt-server && git commit -m "fix: description" && git push	Validate → patch bump → tag → release

New server feature	<code>cd mqtt-server && git commit -m "feat: description" && git push</code>	Validate → minor bump → tag → release
Check client version	<code>cat kitty/Augentix/AugenTix/config.json grep firmware</code>	—
Check server version	<code>cat mqtt-server/package.json grep version</code>	—
View all releases	GitHub → Releases tab	—
Manual release	GitHub → Actions → Run workflow	Select bump type → full pipeline
View CI logs	GitHub → Actions tab	—

10. Troubleshooting

Problem	Cause	Solution
CI didn't trigger	Changed files not in <code>mqtt-client/</code> or <code>kitty/</code>	Only those paths trigger the client CI. Use manual trigger from Actions tab.
CI triggered but no release	Commit message had <code>[skip ci]</code>	Remove <code>[skip ci]</code> from your commit message
Wrong version bumped	Wrong keyword in commit message	Use <code>fix:</code> for patch, <code>feat:</code> for minor
Build failed: <code>cc1</code> not found	Toolchain permissions lost	CI handles this automatically (<code>chmod</code> step)
Build failed: <code>crt1.o</code> not found	New <code>.o</code> file added to toolchain but gitignored	Check <code>.gitignore</code> — <code>!toolchain/**/*.o</code> must be present
Release created but no <code>Augentix.tar.gz</code>	Package step failed	Check Actions log → "Create Augentix.tar.gz" step
Server: tag created but no release	<code>gh release</code> command failed	Check Actions log → "Commit, tag, and release" step
Server: validation passed but no release	It was a Pull Request (releases only on push to main)	Merge the PR → push to main triggers release
Camera doesn't work after OTA	Client version incompatible with server	Check <code>COMPATIBILITY.md</code> in both repos